SOFTWARE DESIGN & IMPLEMENTATION SOLUTION

Digital Banking Loan Approval System

Total Marks: 50

> **Problem Summary:** Design and implement a Python-based Digital Loan Approval System using file handling and modular programming. The system automates application evaluation based on credit score, income, and configurable rules, handles errors gracefully, updates data storage, and generates analytical management reports.

1. System Architecture & Modular Design

The system is designed with high cohesion and low coupling across multiple modules to satisfy all functional requirements, including scalability and extensibility without modifying core logic.

Module Organization

- **`config.json`**: External JSON file containing configurable loan eligibility rules, thresholds, and output paths. Enables smooth extension of new loan types.
- **`input_applications.csv`**: Input file containing applicant records.
- **`evaluator.py`**: Dynamic evaluation module that validates applicant records and evaluates eligibility based on rules.
- **`storage.py`**: Handles reading raw CSV inputs and writing processed applications to standard storage files.
- **`reporter.py`**: Calculates metrics (approval rate, total sanctioned amount) and displays/saves summary reports.
- **`main.py`**: Controller script executing the end-to-end workflow.

2. Configuration & Rule Engine Design (g & h)

To ensure *Extensibility (h)*, rules are decoupled from logic using an external JSON config:

```
// config.json
{
    "loan_rules": {
        "Personal Loan": {
            "min_credit_score": 650,
            "min_income": 30000,
            "max_loan_ratio": 5.0
        },
        "Home Loan": {
            "min_credit_score": 700,
            "min_income": 50000,
            "max_loan_ratio": 8.0
        },
        "Business Loan": {
            "min_credit_score": 720,
            "min_income": 80000,
            "max_loan_ratio": 4.0
        }
    }
}
```

3. Complete Python Implementation

3.1 Application Evaluator & Error Handling Module

```
import json

def load_rules(config_path='config.json'):
    with open(config_path, 'r') as f:
        return json.load(f).get("loan_rules", {})

def evaluate_application(app, rules):
    # f) Error Handling: Validate inputs
    try:
        app_id = app.get('Applicant ID', '').strip()
        name = app.get('Name', '').strip()
        loan_type = app.get('Loan Type', '').strip()

        if not app_id or not name or not loan_type:
            return 'Rejected', 'Missing required applicant details'

        income = float(app.get('Income', 0))
        credit_score = float(app.get('Credit Score', 0))
        requested_amount = float(app.get('Requested Amount', 0))

        if income <= 0:
            return 'Rejected', 'Invalid Income: Income must be positive'
        if not (300 <= credit_score <= 850):
            return 'Rejected', 'Invalid Credit Score: Must be between 300 and 850'
        if requested_amount <= 0:
            return 'Rejected', 'Invalid Requested Amount'

    except (ValueError, TypeError):
        return 'Rejected', 'Invalid numeric format in applicant record'

    # h) Extensibility: Evaluate against dynamic rule parameters
    if loan_type not in rules:
        return 'Rejected', f'Unsupported Loan Type: {loan_type}'

    rule = rules[loan_type]
    reasons = []

    if credit_score < rule['min_credit_score']:
        reasons.append(f"Credit score below threshold ({credit_score} <
{rule['min_credit_score']})")

    if income < rule['min_income']:
        reasons.append(f"Income below threshold ({income} < {rule['min_income']})")

    max_allowed = income * rule['max_loan_ratio']
    if requested_amount > max_allowed:
        reasons.append(f"Loan amount exceeds limit ({requested_amount} > {max_allowed})")

    # c) Decision Processing
    if not reasons:
        return 'Approved', 'All criteria satisfied'
    else:
        return 'Rejected', '; '.join(reasons)
```

3.2 Data Input & Storage Module

```
import csv

# a) Data Input Module
def read_applications(file_path):
    applications = []
    with open(file_path, mode='r', newline='', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        for row in reader:
            applications.append(row)
    return applications

# d) Data Storage Module
def save_processed_applications(file_path, processed_data):
    fieldnames = ['Applicant ID', 'Name', 'Income', 'Credit Score',
                  'Loan Type', 'Requested Amount', 'Status', 'Reason']
    with open(file_path, mode='w', newline='', encoding='utf-8') as f:
        writer = csv.DictWriter(f, fieldnames=fieldnames)
        writer.writeheader()
        writer.writerows(processed_data)
```

3.3 Main Workflow & Report Generation Module

```
def generate_reports(processed_data):
    # e) Report Generation Module
    total_apps = len(processed_data)
    approved = [app for app in processed_data if app['Status'] == 'Approved']
    rejected = [app for app in processed_data if app['Status'] == 'Rejected']

    total_approved_count = len(approved)
    total_rejected_count = len(rejected)
    total_sanctioned_amount = sum(float(app['Requested Amount']) for app in approved if
app['Requested Amount'].replace('.', '', 1).isdigit())
    approval_rate = (total_approved_count / total_apps * 100) if total_apps > 0 else 0.0

    print("=" * 50)
    print("       DIGITAL BANKING LOAN APPROVAL REPORT       ")
    print("=" * 50)
    print(f"Total Applications Processed : {total_apps}")
    print(f"Approved Applications        : {total_approved_count}")
    print(f"Rejected Applications        : {total_rejected_count}")
    print(f"Total Amount Sanctioned      : ${total_sanctioned_amount:,.2f}")
    print(f"Overall Approval Rate        : {approval_rate:.2f}%")
    print("=" * 50)

def main():
    rules = load_rules()
    raw_apps = read_applications('input_applications.csv')

    processed_list = []
    for app in raw_apps:
        status, reason = evaluate_application(app, rules)
        app_copy = app.copy()
        app_copy['Status'] = status
        app_copy['Reason'] = reason
        processed_list.append(app_copy)

    save_processed_applications('processed_applications.csv', processed_list)
    generate_reports(processed_list)
```

```
if __name__ == '__main__':
    main()
```

4. Sample Input & Processed Output

Below is an example execution trace showing input records and system processing output:

Sample Input CSV (input_applications.csv)

Applicant ID	Name	Income	Credit Score	Loan Type	Requested Amount
APP101	John Doe	65000	750	Home Loan	400000
APP102	Jane Smith	25000	620	Personal Loan	50000
APP103	Alice Brown	90000	740	Business Loan	300000
APP104	Bob Wilson	-5000	700	Personal Loan	10000

Processed Output CSV (processed_applications.csv)

Applicant ID	Name	Status	Reason
APP101	John Doe	**Approved**	All criteria satisfied
APP102	Jane Smith	**Rejected**	Credit score below threshold (620 < 650); Income below threshold
APP103	Alice Brown	**Approved**	All criteria satisfied
APP104	Bob Wilson	**Rejected**	Invalid Income: Income must be positive

5. Evaluation of Key Non-Functional Requirements

g) Scalability

For large datasets (hundreds of thousands of records), stream-processing using generator pipelines (e.g., `csv.DictReader` yielding records line-by-line) ensures constant memory overhead O(1) instead of loading full arrays into RAM. Batch database writes or chunked file buffers can be used for persistent storage.

h) Extensibility

Adding a new loan scheme (e.g., `"Education Loan"`) or adjusting minimum credit scores requires zero python code changes. Administrators simply update `config.json`, which is parsed dynamically at runtime.